

Organizing Intelligence Over Time: Human–AI Collaboration as Joint Cognitive Development

Zhiyu Yang¹, Haoyu Wang², Hanqing Wang³, Hao Yang⁴, Xupeng Zhang⁵

¹The University of Texas at Dallas, Richardson, TX, USA

²Nanyang Technological University, Singapore

³Shanghai University of Finance and Economics, Shanghai, China

⁴Sun Yat-sen University, Shenzhen, China

⁵Dark Horse Technology Group Co. Ltd, Beijing, China

zhiyu.yang@utdallas.edu, HA0YU013@e.ntu.edu.sg, whq@163.sufe.edu.cn

yangh627@mail2.sysu.edu.cn, lagelangpeng@gmail.com

Abstract

As humans and AI agents work together across long-horizon tasks and accumulate interaction history, how cognitive work is organized on one task can shape both their later performance and later capability. Within a Human–AI system, the next step may involve further reasoning, retrieval from prior experience, verification, a tool call, another model, human judgment, or stopping. The cognitive operation used also shapes who gets what experience: whether the human continues to practice a skill, whether the AI receives demonstrations or corrections, which failures are exposed for supervision, and which trajectories become reusable. The way cognitive work is organized today therefore helps construct the Human–AI system that will act tomorrow.

This survey connects human cognition, Human–Automation and HCI, and modern AI agents to explain how present cognitive organization shapes future Human–AI capability. Human cognition research explains how limited attention, computation, and control are organized, and how experience can restructure later cognition. Human–Automation and HCI research shows how functions, authority, and initiative are divided across people and machines, and how that division changes human readiness and skill. In modern AI agents, many of these cognitive operations are explicit and programmable, while interaction trajectories can be retained and transformed into memories, workflows, skills, and policies that shape later behavior. Read together, these literatures expose a causal sequence: organizing cognitive work determines the distribution of practice, supervision, feedback, and trajectories; that experience changes human skill and beliefs, machine memory and policy, and the coordination between them; and the updated joint system then organizes later work differently.

We argue that Human–AI collaboration should therefore be studied as a process of *joint cognitive development*. The design problem is to organize current cognition while accounting for the Human–AI system that the current choices are shaping. The way humans and AI work together today changes what each can do tomorrow.

Keywords: Human–AI collaboration; AI agents; longitudinal learning; continual learning; metareasoning; agent memory; self-evolving agents

1 Introduction: From Current Tasks to Future Teams

Consider a coding agent facing an unfamiliar failure. It can reason for another minute, retrieve a similar fix, run a diagnostic test, call a stronger model, ask an experienced developer, or stop. The system is deciding which cognition to invoke, who should provide it, what information it should use, and how much effort or computation to spend. A verification call, a human review, a model switch, and a decision to stop are therefore different ways of organizing the cognitive work required by the same task.

If only the current episode matters, these alternatives can be compared by familiar quantities such as expected accuracy, latency, human effort, or monetary cost. Asking the developer may be expensive but

reliable; another model call may be cheaper; a diagnostic test may reduce uncertainty; stopping may be sensible when further work is unlikely to change the answer. Much of current AI control operates at this timescale: given the capabilities and costs available *now*, select the action that best serves the present task.

When humans and AI work together across tasks, a second timescale appears. The same choice can change the capabilities, beliefs, and evidence available *later*. If the agent handles familiar cases autonomously, it accumulates its own trajectories while the human receives less practice. If the agent acts but a human reviews the result, the human remains engaged and the machine may receive corrections that improve later behavior. If the agent escalates before acting, more of the cognitive work shifts to the human, potentially producing demonstrations or judgments that the machine can reuse. If the human takes over entirely, the current task may become safer or more reliable, but the AI can only reuse that intervention if the relevant actions and reasoning are captured. How work is organized now therefore affects not only what happens on the current task, but also what humans and AI are able to do later.

This creates a tension between current task performance and future capability of the joint Human–AI system. The organization that is suitable now may not be the one that best preserves human skills, improves machine behavior, or strengthens Human–AI collaboration in the future.

Our position. Human–AI collaboration should be studied as a process of joint cognitive development. How cognitive work is organized determines not only the current outcome, but also what experience the human and the AI receive. That experience changes human skill, machine capability, and the coordination between them, which changes how later work should be organized. The central design problem is therefore how to steer the development of the joint system over time.

This position connects three research traditions that have usually been studied separately. Research on *human cognition* contributes two core ideas. First, limited cognitive resources must be selectively spent: a person cannot inspect every alternative, perform every computation, or attend to everything at once (Simon, 1955; Nelson and Narens, 1990; Shenhav et al., 2013). Second, experience changes how those resources need to be spent later: experts may represent a problem differently, retrieve a familiar pattern, proceduralize a routine, or avoid branches of search that a novice would still have to explore (Chase and Simon, 1973; Anderson, 1982; Logan, 1988).

Human–Automation and HCI extend the problem across human–machine boundaries. Function allocation, supervisory control, adaptive automation, and mixed initiative ask how functions, authority, information, and initiative should be divided between people and machines (Fitts, 1951; Sheridan and Verplank, 1978; Parasuraman et al., 2000; Horvitz, 1999). Human-factors research also shows that this division changes the human who remains in the system: automation can alter practice, situation awareness, readiness, and the ability to recover from failure (Bainbridge, 1983; Endsley and Kiris, 1995). The organization of current work can therefore change the future human participant.

Modern AI adds an adaptive machine participant whose task-time behavior and cross-task learning can both be made explicit. Long-horizon and human-in-the-loop agents expose cognitive control in executable form, while agent memory and experiential-learning systems can turn trajectories into workflows, skills, rules, or reusable state (Luo et al., 2026). Continual and self-evolving agents can further use accumulated experience to change later behavior (Shinn et al., 2023; Wang et al., 2025; Li et al., 2026; Wu et al., 2025; Silver and Sutton, 2025). Test-time compute and model routing likewise make the expenditure of machine cognition directly controllable (Snell et al., 2024; Zhang et al., 2026). What was once largely implicit in human cognition or fixed in automation can increasingly be logged, measured, reused, and adapted on the machine side.

Recent work from frontier AI laboratories makes this convergence concrete. In *Research Acceleration: The View Inside OpenAI*, OpenAI describes researchers delegating increasingly long-horizon tasks to concurrent agents while continuing to set research priorities and judge which results deserve further pursuit (OpenAI, 2026). Anthropic’s *Effective Harnesses for Long-Running Agents* shows how progress records and structured

handoffs support work across sessions, while *Scaling Managed Agents: Decoupling the Brain from the Hands* makes durable session state a first-class object outside both the model’s context window and agent harness, allowing long-horizon work to survive changes in the machinery executing it (Anthropic, 2025c, 2026). In *From Atari to EVE Online: Building on 15 Years of AI Research in Games*, Google DeepMind identifies continual learning, memory beyond current context windows, and planning over weeks, months, or years as goals for agents operating in persistent worlds (Moufarek and Bolton, 2026). As agents undertake longer stretches of work and retain more of their execution history, how that experience should change subsequent reasoning, verification, and human involvement becomes a concrete research question.

Now that both human and the AI can contribute cognition to the task and learn from how the work is carried out. Work organization becomes part of the learning process itself, shaping who learns and what each side gets to learn from, leading us to the loop studied in this survey: *organize cognitive work* → *generate selective experience* → *change the human, the AI, and their coordination* → *reorganize future work*.

We organize the survey around three questions:

1. **How is cognitive work organized within a task?** Which cognitive operations are invoked, by whom, and for how long?
2. **How does experience change the cognition available next time?** Which representations, procedures, skills, and control policies persist into later tasks?
3. **How does organizing work change the learners themselves?** How do practice and supervision shape future human, machine, and joint capability?

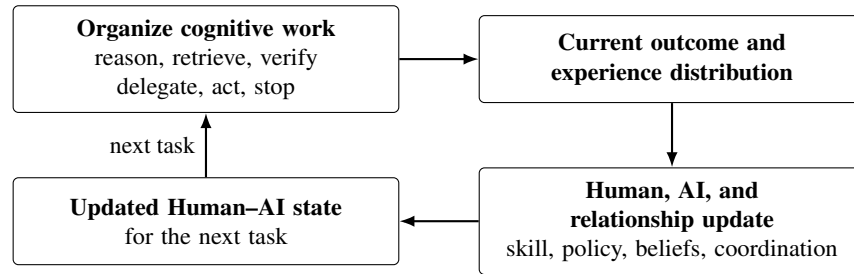


Figure 1: The joint cognitive development loop. In a Human–AI system, organizing cognitive work determines not only the current outcome but also the distribution of practice, supervision, feedback, and trajectories. That experience changes the human, the AI, and their relationship, producing an updated joint system that organizes later work differently.

2 How Is Cognitive Work Organized Within a Task?

Section 2 asks how a Human–AI system should organize cognition on the current task before experience changes either participant. The literatures below approach this question from increasingly broader levels: what information should enter the problem, which computations are worth performing, how effort should be regulated, which actor should perform a function, where authority should lie, and when another actor should be brought in. Modern AI systems increasingly place these decisions within the same interactive setting, making it useful to read these traditions together as different accounts of how cognitive work should be organized on a task, who should handle each part, and how much effort should be spent.

2.1 Bounded Rationality: Deciding What Enters the Problem

Bounded rationality makes selection itself part of cognitive organization: a limited decision maker must decide what information, alternatives, and search branches enter the problem at all. Simon’s formulation starts from limits on time, information, memory, and computation (Simon, 1955; Griffiths et al., 2015; Lieder and Griffiths, 2020). A bounded system cannot simply take a fully specified decision problem and solve it to optimality; it must first reduce the problem to something it can handle.

Consider apartment search. A hypothetical optimizer could enumerate every viable apartment in a city, estimate rent, commute, safety, amenities, uncertainty, and future changes, and compare them all. A real search looks very different. Someone may restrict attention to a few neighborhoods, inspect only the listings that appear on two websites, ask several friends, and stop once an apartment satisfies a set of aspiration levels. Simon called this kind of behavior *satisficing*: search ends when an alternative is good enough rather than when global optimality has been established.

What matters for our purposes is where the constraint enters. Limited cognition affects the problem *before* the final action is chosen. It determines which alternatives receive attention, what information is collected, how finely the situation is represented, and how long search continues. The first problem of organizing cognition is therefore deciding what becomes cognitively active at all (Ho et al., 2022).

This point has a direct analogue in modern AI. A model with a large context window still does not benefit equally from every possible document, memory, branch of search, or intermediate state. Retrieval, context selection, search breadth, and stopping all determine which part of a much larger problem the system actually processes.

2.2 Metareasoning: Deciding Which Computation Is Worth Performing

Rational metareasoning makes internal computation itself a selectable action. Russell and Wefald ask not only what an agent should do in the external world, but also which computation it should perform before acting (Russell and Wefald, 1991). Treating internal computation as an action makes reasoning, search, information gathering, verification, and stopping selectable cognitive organization.

Their central concept is the *value of computation*. A computation is useful when the improvement it is expected to produce in the eventual decision is worth its cost. If examining one uncertain alternative could plausibly change which action is selected, the computation may be valuable. Further refining an estimate that is already unlikely to affect the decision may not be.

Stopping follows from the same principle. Reasoning should continue while some available computation has sufficient expected value; it should stop when additional computation is unlikely to justify its cost. The issue is therefore not whether more reasoning is generally good or bad. It is whether a particular computation is likely to gain value for the decision at hand.

This computation-budgeting problem has a direct analogue in modern AI. Additional reasoning tokens, another search branch, a verifier call, a stronger model, or a diagnostic tool all consume resources in exchange for a possible improvement in the answer. Test-time computation is therefore not only a question of how much compute is available, but which computation is worth its cost on this instance.

Metareasoning introduces a further difficulty: deciding whether another computation is worthwhile can itself require computation. Exact metalevel optimization can therefore consume the resources it is supposed to manage. Later work treats this as a learning problem, asking whether experience can teach an agent which cognitive strategy is appropriate for which situation (Lieder and Griffiths, 2017; Callaway et al., 2018). Section 3 examines how experience can change the policy that selects which cognitive operations to invoke.

2.3 Monitoring and Control: Deciding When More Cognition Will Help

Metacognitive monitoring and control ask how a system regulates further cognition when the value of additional effort is not directly observable.

Nelson and Narens formalize metacognition as two coupled levels (Nelson and Narens, 1990). The object level performs the task itself, while a meta level maintains an estimate of how that process is going. Information flowing upward supports monitoring; control flows downward when the meta level uses that estimate to continue, stop, or change the ongoing cognitive process. The meta level cannot directly observe how well the object-level process is actually doing, so it relies on cues such as confidence, fluency, perceived difficulty, or progress (Koriat, 1997). Monitoring uses these cues to estimate the state of cognition; control uses that estimate to continue, switch strategy, spend more time, or stop.

But a monitoring signal does not by itself determine the appropriate control response. Son and Metcalfe found that people may devote more study time to difficult material under some conditions, but under severe time pressure may instead favor easier items that offer better returns per unit time (Son and Metcalfe, 2000; Metcalfe and Kornell, 2005). “This is difficult” is therefore not itself a control policy. The relevant question is whether further effort is likely to improve the outcome enough to be worthwhile.

The Expected Value of Control framework explains why the same difficulty signal can lead to different amounts of effort: control depends on expected reward, the efficacy of additional control, and its cost (Shenhav et al., 2013, 2017; Froemer et al., 2021; Masis Obando et al., 2025). A very difficult task may deserve little further effort if more effort is unlikely to change the result. A moderately difficult task may deserve substantial effort if another unit of cognition has a good chance of helping.

Modern AI systems face the same monitoring and control problem. A verifier score, model confidence, or execution signal estimates how well the current attempt is going. The system then uses that estimate to decide whether to continue reasoning, verify again, escalate to a human, or stop. As in human metacognition, the signal alone does not determine the action. What matters is whether additional cognition is expected to improve the result enough to justify its cost.

2.4 Function Allocation and Supervisory Control: Deciding Who Performs the Cognitive Work

Function allocation extends cognitive organization across human–machine boundaries. Once multiple actors can contribute, the problem includes both what cognitive operation should occur and who should perform it.

This broader view has long treated human and machine capabilities as parts of a coupled cognitive arrangement: Licklider described human–computer symbiosis, while distributed-cognition research showed how cognition can be organized across people, representations, and artifacts (Licklider, 1960; Hutchins, 1995).

Early function-allocation work framed this as a division of labor between people and machines. Fitts’s 1951 report framed function allocation as decomposing a system into functions, comparing human and machine capabilities, and assigning each function accordingly (Fitts, 1951; Hollnagel and Bye, 2000).

Sheridan and Verplank made this organization more dynamic in their work on supervisory control (Sheridan and Verplank, 1978). Under direct control, a person continuously manipulates the machine and closes the low-level feedback loop. Under supervisory control, local automation handles faster or lower-level operations while the human works at a different level: specifying goals, issuing commands, monitoring execution, and intervening when necessary. Cognitive work is distributed not only across actors, but also across levels of abstraction and timescales (Hollnagel and Woods, 2005).

Their levels of automation further show why “human or machine” is too coarse a distinction. Labeling a task as either human or automated hides important differences in how the work is actually divided. A computer may generate alternatives while the human chooses among them, recommend an action while the

human retains approval, or execute the action while the human retains veto or oversight.

Parasuraman, Sheridan, and Wickens make the division of human and machine work more precise by distinguishing the *type* and *level* of automation (Parasuraman et al., 2000). Information acquisition, analysis, decision selection, and action implementation can each be automated to different degrees. A system may detect a problem automatically but leave interpretation to the human; another may interpret the situation and recommend an action while leaving execution under human control.

Once a system can draw on human judgment, model reasoning, and external tools, it must determine not only what cognitive operation should occur, but who should perform it and who retains authority over what happens next. Model routing, tool use, human escalation, and human approval in modern AI are contemporary instances of this problem.

2.5 Mixed Initiative: Deciding When to Invoke Another Actor and What to Show

Mixed-initiative interaction allows initiative to shift between the human and the machine as the task unfolds. The timing of human intervention or machine action therefore need not be fixed in advance.

Horvitz treats human involvement as a decision under uncertainty and interruption cost (Horvitz, 1999). When the system is uncertain, it must weigh the value of human input against the cost of demanding attention. Human attention therefore becomes another scarce resource in the organization of cognitive work. Later work on adjustable and shared autonomy similarly allows control to shift during execution rather than being assigned once in advance (Scerri et al., 2002; Javdani et al., 2018).

The same problem appears in modern Human–AI systems. An agent deciding whether to ask a developer for help must determine whether human judgment is worth the interruption, what evidence to present, and whether the human should advise, verify, approve, or take over. Mixed initiative therefore adds a temporal question to function allocation by asking when human or machine cognition should enter the task.

2.6 Modern AI Brings These Problems Together

Modern agent runtimes can now coordinate what cognitive work is done, how it is divided across models, tools, and humans, and when additional cognition is worth invoking. Context construction determines what information is active; test-time reasoning and verification determine what computation is performed; routing determines which model or tool contributes; escalation determines when human cognition enters; and stopping determines when further work ends. These controls interact: retrieval can reduce the need for reasoning, verifier disagreement can justify escalation, and a trusted procedure can make both unnecessary.

Frontier implementations expose these decisions as distinct mechanisms. Anthropic’s context-engineering account combines selective retrieval, compaction, external notes, and specialized subagents to control which information remains active (Anthropic, 2025a). Its long-running coding harness uses an initializer, a feature list, progress records, and incremental implementation to coordinate successive sessions (Anthropic, 2025c). Co-Scientist uses a supervisor to coordinate hypothesis generation, reflection, pairwise ranking, evolution, and meta-review; its reported emphasis on verification concerns comparison against literature and data, which remains distinct from experimental confirmation (Co-Scientist Team, 2026). These designs make context, decomposition, continuation, and evaluation inspectable parts of task execution.

The environment also determines which judgments can be delegated. OpenAI’s harness-engineering report describes moving repository knowledge into agent-accessible documentation and encoding recurring quality requirements in checks and cleanup processes (Lopopolo, 2026). Human judgment can thus enter before a particular intervention, through the construction of tools, tests, and constraints that later agents use. A runtime-level account should include this preparation and maintenance effort alongside visible model calls and human approvals.

Variables studied separately in these traditions can now be configured together in a single runtime. This

brings several modern AI problems under a common within-task question: *given limited and heterogeneous forms of cognition, what should be brought to bear on the current task?*

That question cannot be reduced to a single “compute budget.” More reasoning tokens, a retrieval call, a verifier, a stronger model, and ten minutes of expert attention differ not only in cost, but also in the information and error correction they provide. Organizing cognitive work therefore requires deciding which form of cognition is useful for the current task, not merely how much total resource to spend.

So far, however, this is still a within-task problem. It assumes that the available cognitive resources and the policies for using them are given when the task begins. The next section removes that assumption. Experience can change the representation, procedure, skill, memory, and metalevel policy that will be available on the next task.

3 How Does Experience Change the Cognition Available Next Time?

Section 2 asked how cognition should be organized within a task, without yet asking how experience changes the capabilities available on later tasks. Section 3 turns to that question. Experience can change cognition itself by turning recurring structure into chunks, procedures, or retrievable instances that simplify later tasks. It can also change how cognition is controlled by altering which operations are selected: when retrieval is sufficient, when deliberate reasoning is invoked, when verification is triggered, and when help is sought.

3.1 Experience Creates Reusable Structure for Later Tasks

Experience can change representation before deliberate reasoning begins. Chase and Simon’s chess studies show this clearly (Chase and Simon, 1973; Gobet and Simon, 1996): when skilled players briefly viewed meaningful chess positions, masters could reproduce far more pieces than novices, but much of the advantage disappeared for randomly arranged pieces. The difference was not simply a larger general-purpose memory; experts had learned domain structure that novices did not perceive.

A *chunk* groups lower-level elements into a meaningful unit. A novice may encode several pieces separately; an expert may recognize a familiar configuration of threats, defenses, and spatial relations. Deliberate reasoning begins from a different state representation. The expert is not merely searching the novice’s problem faster; experience has changed the problem that enters search.

Anderson describes a related change in procedure (Anderson, 1982). A novice may need to retrieve a declarative rule, interpret it, bind it to the current case, and then execute it. With practice, some of these intermediate steps can be proceduralized into productions that respond more directly to familiar conditions. The same task can therefore require fewer explicit cognitive steps after practice.

Logan’s instance theory gives yet another route (Logan, 1988). Repeated encounters store specific instances. On a later task, direct retrieval can compete with a general algorithm; as relevant instances accumulate, retrieval increasingly wins. A familiar result may simply come to mind before the general procedure finishes.

The mechanisms differ, but their consequence for organizing cognition is the same. Experience changes what needs to happen on the next encounter. It can alter the representation presented to reasoning, replace an interpreted rule with a procedure, or replace fresh calculation with retrieval (Schneider and Shiffrin, 1977; Shiffrin and Schneider, 1977). The available cognitive operations are therefore not fixed across tasks.

3.2 Recognition Lets Familiar Cases Bypass Search

Recognition-primed decision making shows how experience can reduce the amount of search that must be executed at all. Experts often do not begin by generating and comparing a large set of alternatives. A familiar

situation can evoke a plausible action directly; the expert checks whether it is likely to work and searches for another option only if necessary. (Klein et al., 2010)

Experience is doing more than speeding up search. It is changing when search begins, which candidates enter it, and how far it proceeds. Familiar structure allows some branches of cognition to be skipped entirely.

But recognition is useful only when past experience is informative about the present case. Kahneman and Klein argue that reliable intuition depends on environments with learnable regularities and feedback good enough to connect cues with outcomes (Kahneman and Klein, 2009; Bilalić et al., 2008). When the environment is unstable or feedback is weak, familiarity can produce confidence without reliable prediction. A firefighter may recognize a familiar situation and act quickly, but an unexpected cue can signal that the familiar pattern no longer applies and prompt a return to deliberation.

3.3 Experience Changes Which Cognitive Operations Are Chosen

Experience can change the meta-level policy that selects among cognitive operations. Instead of only providing a better representation, procedure, or retrieved answer, past experience can also change which cognitive operation the system chooses in a given situation.

This possibility already appears in rational metareasoning. If an agent has to solve a costly metalevel optimization every time it decides how to think, then metareasoning itself can become the bottleneck. Lieder and Griffiths treat strategy selection as something that can be learned across problems: experience with previous decisions can teach which cognitive strategy tends to work well in which situations (Lieder and Griffiths, 2017; Callaway et al., 2018). The agent no longer has to derive its strategy from scratch on every task.

More recently, Silver and Sutton argue that experiential agents should use accumulated experience to improve the methods they use to reason and act across tasks (Silver and Sutton, 2025).

Modern agent systems support both object-level reuse and meta-level control. Persistent memory and experiential reuse already span several mechanisms: Generative Agents retain observations for reflection and planning, ExpeL derives reusable insights from successes and failures, and Voyager accumulates executable skills during open-ended interaction (Park et al., 2023; Zhao et al., 2024; Wang et al., 2023). Some systems more directly change the object-level route through a task. Agent Workflow Memory turns previous trajectories into reusable procedures (Wang et al., 2025); Reason Wide distills recurring reasoning patterns into skills that can be invoked later (Singh et al., 2026); Trace2Skill and SkillRL make trajectory-to-skill transfer explicit (Ni et al., 2026; Xia et al., 2026). In these cases, past work changes what computation is available on the next task.

Other systems use experience to change how computation is selected. JitRL retains state–action–return evidence that can bias later action selection (Li et al., 2026). EvoRoute uses accumulated quality, latency, and cost information to change which model is chosen (Zhang et al., 2026). Experience-driven early termination uses previous resolution trajectories to estimate whether continued work is likely to pay off (Guo et al., 2026). EvolveR, Evolving-RL, and the ELL framework make the extraction and use of experience part of a coupled self-improvement loop (Wu et al., 2025; Fan et al., 2026; Cai et al., 2025). Here experience changes decisions about routing, continuation, and expenditure rather than only supplying reusable task content.

Recent work makes the boundary between reusable content and revisable control more explicit. Anthropic’s Agent Skills package instructions, scripts, and resources for selective loading; this is an interface for procedural reuse, not evidence that the skills were learned autonomously (Anthropic, 2025b). Meta Context Engineering goes further: a base agent optimizes context as files and code, while a meta-agent uses execution and evaluation histories to recombine the skills governing that optimization (Ye et al., 2026). The object being revised includes how future context will be constructed.

Prime Agent provides a complementary runtime example. With model weights fixed, its Continual Harness retains prompt notes, memories, executable skills, and subagent specifications, and converts trajectory

evidence into versioned updates available to later invocations (Karten et al., 2026). Retained experience can therefore alter not only task content but also behavioral instructions and divisions of labor. Such mechanisms make cross-task adaptation implementable; demonstrating that their changes improve transfer still requires comparisons against matched histories and resource budgets.

These two forms of learning change different parts of future cognition. A learned workflow changes how the task itself is carried out. A learned routing or stopping policy changes how the system decides what cognition to use while carrying out the task. Experience can therefore be reused both in task execution and in deciding how task execution should be organized.

3.4 Reopening Restores Cognition When Past Experience No Longer Fits

Experience can reduce future cognitive work by turning past regularities into shortcuts. That benefit depends on those regularities continuing to hold. When they no longer do, the same shortcut can suppress reasoning or verification that has become necessary again. A system that learns when cognition can be skipped therefore also needs to learn when it should be reopened.

Research on recognition-based expertise suggests one useful trigger for reopening: violated expectations. In one of Klein et al.’s classic fireground cases, a firefighter noticed that water was not having the expected effect while the burning house became intensely hot but unusually quiet. He ordered his crew to evacuate, and within a minute the floor collapsed over a basement fire. Klein et al. interpret the mismatch between expected and observed cues as the signal that triggered reassessment (Klein et al., 2010). Modern AI needs analogous mechanisms for deciding when a stored workflow, skill, routing preference, memory, or user model no longer deserves to control the current case. Memory revision, skill curation, and controlled-history evaluations already expose related problems of stale experience, interference, and negative transfer (Cao et al., 2026; Ouyang et al., 2026; Yang et al., 2026; Shu et al., 2026). Distribution shift, unusual uncertainty, conflicting evidence, or high stakes may all justify fresh cognition.

Staleness can also arise because the underlying model improves. Anthropic reports that context resets needed to address one model’s premature stopping became unnecessary with a later model, leaving an inherited harness intervention with avoidable overhead (Rajasekaran, 2026; Anthropic, 2026). This is evidence for revisiting engineering assumptions, not a demonstration of a learned reopening policy. It nevertheless exposes a concrete target: reusable control should track the model, tools, and environment under which it was useful, as well as the task pattern it matches.

Experience is useful partly because it lets later cognition be skipped. But that shortcut must remain revocable. When current evidence no longer fits what was learned, the system must be able to return the case to active reasoning rather than continue following stale experience.

Experience can change what humans and AI are able to do later. How earlier work is organized determines what practice, feedback, and supervision each side receives. Section 4 asks how the organization of cognitive work changes the human and the AI over time.

4 How Does Organizing Work Change the Learners?

Section 3 asked how experience changes the cognition available on later tasks. Section 4 asks what determines the experience that each participant accumulates. How work is organized affects who gets to practice, who encounters errors, who receives feedback, and who observes the consequences of a decision. Repeated across tasks, these differences can change human skill, machine behavior, and the coordination between them.

4.1 Automation Changes What the Human Continues to Practice

Human–Automation research established long before adaptive AI that the division of work changes what the human gets to practice. Bainbridge’s “Ironies of Automation” starts from a familiar division of labor: automate routine operations and leave unusual or difficult cases to the human (Bainbridge, 1983; Kaber and Endsley, 2004; Parasuraman and Riley, 1997). That division looks sensible if human capability is treated as fixed. But automating routine operation also removes the practice through which process knowledge is maintained, leaving the human to monitor for long periods and intervene mainly when unusual cases arise. The hardest cases may therefore fall to the operator with the least recent practice.

Endsley and Kiris demonstrated a related effect experimentally (Endsley and Kiris, 1995; Kaber and Endsley, 2004). Participants who had worked with higher levels of automation were slower to resume manual decision making when the automation failed. Their result is usually discussed as the *out-of-the-loop performance problem*: retaining nominal authority does not guarantee that a person remains cognitively prepared to exercise it.

Situation awareness explains why a human can retain formal authority while becoming less ready to intervene. Active control requires the operator to interpret events, anticipate what may happen next, and keep an updated understanding of the system. Passive monitoring may present the same information without requiring that active understanding, leaving the operator formally in control but less prepared to take over when needed.

A contemporary coding experiment tests the learning consequence directly. Shen and Tamkin randomly assigned 52 developers learning the unfamiliar Trio library to work with or without AI assistance. The assisted group averaged 50% on the subsequent mastery quiz versus 67% without assistance; the average completion-time difference was not statistically significant (Shen and Tamkin, 2026). Their qualitative analysis associated explanation-seeking and other engaged interaction patterns with better learning, but those patterns were not themselves randomly assigned. The result concerns immediate skill acquisition in this setting, rather than established long-term deskilling. It nonetheless shows why completed code and the ability to understand or debug that code require separate measurement.

This changes the meaning of function allocation from the previous section. Assigning a function to automation does not merely remove that function from the human’s current workload. It can also remove practice, alter attention, and change the human capability available later.

4.2 Interaction Changes Both Human and Machine Capability

Classical automation already showed that work organization can change the human. Adaptive AI adds a machine that can retain interaction history and update its later behavior.

People also learn how a model behaves over time: where it tends to succeed, where it fails, when its advice deserves checking, and how much effort is worth spending on oversight. These learned expectations become part of future collaboration. Bansal et al. show one consequence: improving a model’s standalone accuracy can still hurt Human–AI team performance when an update breaks patterns of behavior that users had already learned to work with (Bansal et al., 2019; Lee and See, 2004). What the human has learned about the machine is therefore part of the effective capability of the team.

Shared autonomy shows how machine behavior can actively shape later human strategy. Nikolaidis et al. model not only what strategy the human prefers, but also how willing the human is to adapt (Nikolaidis et al., 2017; Lee and See, 2004). The robot can then choose whether to accommodate the person’s current behavior or act in ways that encourage movement toward a different joint strategy. Machine behavior affects the current task, but it can also change how the human behaves in later interaction.

The machine can learn from the human in return. In SARI, human demonstrations improve later autonomous assistance across interactions (Jonnavittula et al., 2024). Personalized agents can retain information

from clarification and correction (Liang et al., 2026). Human-in-the-loop systems can use expert feedback both to complete the present task and to improve later behavior (Yang et al., 2026; Mozannar et al., 2022). Coding agents can turn accepted review feedback into reusable behavioral rules (Aggarwal and Farhady Ghalaty, 2026).

Deployment observations also suggest that oversight changes with experience. Anthropic reports that more experienced Claude Code users both auto-approve more actions and interrupt agents more often (McCain et al., 2026). Fewer approvals therefore need not imply uniformly less supervision: people may change when and how they intervene. These observational patterns do not isolate learning from differences in users, tasks, or model versions. They motivate measuring the quality and timing of intervention rather than using approval frequency alone as a proxy for human engagement.

Human intervention therefore has two roles. It can provide cognition that the current task needs, and it can create experience from which the machine changes. Likewise, machine behavior can solve part of the current task while teaching the human how the system behaves. The interaction history can therefore change both participants before the next task begins.

4.3 Work Organization Determines What Each Side Gets to Learn From

Human–AI work organization determines which experiences become learning opportunities for each side. For the AI, escalation and review policies determine which trajectories receive human feedback. An agent that rarely asks for help will receive fewer corrections or demonstrations than one that escalates more often, even if both encounter the same underlying tasks.

For the human, automation determines which parts of the task remain available for practice. If the AI handles familiar cases autonomously, the human increasingly encounters unusual or difficult ones. If recommendations are presented before the human forms an independent judgment, the human practices responding to AI suggestions rather than solving the task unaided. If intermediate state is hidden, the human may observe outcomes without practicing the reasoning needed to produce them.

Scientific agents bring a further tension into view. DeepMind’s *Conjecture Machines* essay argues that cheaper hypothesis generation can place greater pressure on experimental validation and expert review, and raises concern about the opportunities through which junior scientists acquire judgment (Wallace et al., 2026). This is a prospective diagnosis, not a longitudinal measurement of skill loss. For the present framework, it suggests a testable possibility: shifting experts from producing solutions to screening them can increase demand for judgment while changing how that judgment is learned. Verification capacity and the development of future verifiers must then be studied together.

Later behavior therefore reflects not only what the human and the AI learned, but also what earlier work organization allowed them to learn from. Greater user agreement with AI recommendations may reflect calibrated trust, dependence, or adaptation to the system. Greater AI autonomy may reflect improved capability, or a task distribution from which difficult cases were repeatedly escalated away. How work is organized today therefore shapes both the experience available for learning and the behavior observed later.

4.4 Closing the Loop: Changes in the Learners Reshape Future Work Organization

The within-task organization of cognition, cross-task reuse of experience, and selective distribution of practice and supervision now form a closed longitudinal loop. Each task is handled by a Human–AI system partly shaped by earlier work, and its current work organization helps shape the system that will handle the next one.

We use *Human–AI co-evolution* for this recursive condition. The changes must persist beyond a single interaction and arise partly from the other participant or the way work is shared. They must also feed back into how later work is performed. Human learning changes future interaction with the AI; machine learning

changes the environment to which the human subsequently adapts; both changes affect what experience will be generated next.

This feedback makes short-term performance and long-term development different objectives. A work policy can improve the current task while reducing human practice or machine learning opportunities, or accept a short-term cost to build future capability.

The evidence reviewed here establishes components of this loop more strongly than the complete loop. Runtime reports show revisable work organization; agent studies test selected forms of experience reuse; human studies measure learning or changing interaction patterns. Demonstrating joint cognitive development requires linking these changes within the same history, rather than inferring reciprocal improvement from their separate existence. The research agenda below targets that missing longitudinal connection.

The important unit of analysis is therefore a joint cognitive system whose participants change through their history of work. How humans and AI work together today changes what each can do tomorrow.

5 Research Questions for Joint Cognitive Development

Modern AI systems already adapt memory, skills, routing, stopping, test-time computation, and human involvement. The harder questions arise when these mechanisms share a history, when current work determines what future evidence will exist, and when both the human and the AI change through that history. The six questions below target the control, learning, and identification problems that arise when present work shapes future experience and both the human and the AI adapt over time.

5.1 How Should Interdependent Cognitive Operations Be Learned Together?

Experience-conditioned cognitive control asks whether prior trajectories can guide how cognition is organized on the next task. Existing systems already control fragments of this problem. Snell et al. and CoBa adapt inference-time computation, while EvoRoute and EET adapt routing or stopping (Snell et al., 2024; Zhou et al., 2026; Zhang et al., 2026; Guo et al., 2026). JitRL uses trajectory evidence to modulate later action probabilities; MCMA, SkillIOS, and JERP adapt memory, skills, or experiential rules (Li et al., 2026; Liang et al., 2026; Ouyang et al., 2026; Ye and Yu, 2026). Agent Workflow Memory, Reason Wide, Trace2Skill, and SkillIRL turn trajectories into reusable procedures or skills (Wang et al., 2025; Singh et al., 2026; Ni et al., 2026; Xia et al., 2026). Learned deferral and complementary-team policies provide related mechanisms for choosing among machine and human contributors (Madras et al., 2018; Mozannar and Sontag, 2020; Wilder et al., 2020; Gao et al., 2023; Lykouris and Weng, 2025).

Modern agents often control retrieval, reasoning, model routing, verification, human involvement, and stopping separately. The problem is that these decisions are not independent. Retrieving a reliable workflow may remove the need for further reasoning or verification, while an unfamiliar case with conflicting evidence may make a stronger model, additional verification, and human review valuable together. A controller that decides each operation in isolation can therefore spend cognition where another operation has already made it unnecessary, or stop before signals from another part of the system have been taken into account.

A tractable starting point is to keep the base model fixed and learn a policy over a bounded set of runtime choices: retrieve a procedure, perform a diagnostic, invoke a verifier, ask for review, or continue independently. MCE’s revisable context-engineering skills and Prime Agent’s persistent control artifacts provide concrete representations for parts of this design space (Ye et al., 2026; Karten et al., 2026). We propose evaluating whether jointly adapting a few interacting choices outperforms adapting each separately under the same total learning and execution budget. This would test coordination directly, without requiring an unconstrained system to rewrite its entire runtime.

A central research question is therefore to learn from experience how these cognitive operations should be

coordinated. Rather than deciding each operation independently on every task, an agent could learn which combinations have worked in similar situations and reuse those strategies on later tasks. The challenge is to learn such a policy from trajectories, including which parts of a successful or failed strategy deserve credit, without having to solve the entire organization problem from scratch each time.

5.2 Reopening: When Should Experience Lose Authority?

Reusable experience gains authority by suppressing work that would otherwise be performed. A workflow can replace search, a skill can replace fresh reasoning, a routing preference can exclude models, and a user model can determine which questions are never asked. MCMA, SkillIOS, and ReMe already make memory, skill management, or revision active control problems, while PATH-Bench and AgentCL expose interference, negative transfer, and forgetting under controlled histories (Liang et al., 2026; Ouyang et al., 2026; Cao et al., 2026; Yang et al., 2026; Shu et al., 2026). What remains less developed is deciding when a previously useful artifact should stop controlling the current case.

A concrete failure occurs when a routing preference reflects an obsolete model version or price, yet continues to bypass alternatives because its own past success prevents new comparative evidence from being collected. The same self-sealing pattern appears when a familiar workflow suppresses the verification that would reveal its assumptions are violated. Retrieval confidence alone cannot resolve this problem: an artifact can be highly similar to the present case and still rely on a regularity that has changed.

A central research question is a reopening policy attached to reusable experience. Each workflow, skill, memory, routing preference, or user model should carry evidence about its domain of validity, provenance, relevant environmental assumptions, and observations that would weaken it. A reopening decision should consider anomaly, contradiction, drift, stakes, and the cost of being wrong. The system can then use the artifact directly, down-weight it, test it, or revoke it. Later outcomes should update both the artifact and its authority boundary. This turns “when should the agent reason again?” into an explicit revocation and change-detection problem rather than a generic confidence threshold. Reopening also requires evidence that routine reuse would otherwise suppress. One experimental design would introduce controlled changes to tool behavior, model versions, or task regularities after a procedure has become useful. Compare immediate reuse, periodic revalidation, and evidence-triggered reopening on detection delay, unnecessary reopening, downstream error, and recovery cost. Versioned artifacts permit rollback, but deciding when to roll back remains a separate inference problem. Occasional checks on apparently familiar cases can reveal whether the policy is merely becoming confident in its own selectively observed history.

5.3 How Should Work Be Assigned When It Also Creates Learning Opportunities?

Human–AI task assignment determines both who solves the current problem and who receives the resulting learning opportunity. Most routing and delegation methods score an assignment by current accuracy, latency, monetary cost, compute, or human effort. Once work changes future capability, that objective is incomplete: automation can remove routine practice that sustains human readiness (Bainbridge, 1983; Endsley and Kiris, 1995; Kaber and Endsley, 2004; Parasuraman and Riley, 1997), while human intervention can provide demonstrations or corrections that improve later machine behavior (Jonnavittula et al., 2024; Yang et al., 2026).

A locally optimal assignment can therefore create a developmental trap. Routing every difficult case to an expert produces reliable answers but keeps the machine’s experience concentrated on easy cases. Automating every routine case improves throughput but leaves the human responsible for rare failures without the practice needed to understand them. Random exploration is inadequate because the costs of failure, human fatigue, and learning differ across tasks and participants.

A central research question is a work-assignment policy that maintains learner states for both participants

and values an assignment by immediate utility plus its expected effect on future capability. The learner-state representation must distinguish machine uncertainty from machine learning potential, and human workload from skill maintenance or acquisition. A useful formulation would choose among autonomous, human, and joint execution, including when to request demonstration, review, or takeover, and then update estimates of what each participant learned. The central estimation problem is the future value of experience: which participant should perform this task because the resulting trajectory or practice will most improve the joint system on later tasks? For coding, an actionable intervention is to vary whether the developer attempts a diagnosis before seeing a generated fix, requests an explanation after generation, or delegates the entire repair. The subsequent probe should measure unaided debugging and intervention on a new failure as well as assisted task performance. The observed interaction patterns in Shen and Tamkin (2026) motivate these treatments; they do not establish their causal effects. Randomizing assistance modes over a task sequence would test which forms of participation preserve useful practice while still generating machine-readable feedback.

5.4 How Should Systems Learn When Their Own Decisions Determine the Feedback They Receive?

A system may learn from human corrections, execution outcomes, and other signals produced during use, but decisions about autonomy, review, routing, and takeover also determine which of those signals become available for later learning. Supervision is therefore endogenous: the system’s current organization of work helps produce the feedback from which it will later learn. Systems such as HILA, JitRL, JERP, Agent Lightning, and learned expert-intervention policies already learn from feedback generated during operation (Yang et al., 2026; Li et al., 2026; Ye and Yu, 2026; Luo et al., 2025; Wang et al., 2026), but they leave open the broader problem of learning from data whose availability depends on the system’s own behavior.

The resulting training history can be systematically incomplete. If humans review only low-confidence cases, confident errors are less likely to receive corrections. If a human takes over before a risky plan is executed, the AI never observes what would have happened had it continued. If successful autonomous cases receive no explicit feedback, silence cannot be safely interpreted as evidence of correctness. Learning directly from such a history can reinforce existing blind spots because the system receives more feedback where its current policy already chooses to seek it.

A central research question is therefore how to learn from operational feedback while accounting for how that feedback was generated. Systems may need to record why review, routing, or takeover occurred, distinguish missing outcomes from favorable ones, and deliberately obtain feedback from cases that the current policy would otherwise ignore. Randomized audits, safe exploration, counterfactual estimation, and explicit logging of interrupted trajectories offer possible starting points. The goal is to improve from experience without allowing the current organization of work to systematically hide failures from later learning. Feedback quality also depends on the evaluator’s own development. Anthropic’s application-development experiments separate generator and evaluator roles and calibrate an evaluator against explicit criteria because naive self-assessment was overly favorable (Rajasekaran, 2026). Our proposed extension is to record evaluator versions and periodically compare their judgments with independent outcomes. Otherwise, an apparent learning curve can arise because the agent learns an evaluator’s preferences, or because the evaluator becomes more permissive. The remaining human review budget should be measured together with the number of candidates awaiting validation.

5.5 How Can a System Distinguish User Preference from Adaptation to the System?

A long-term assistant may interpret consistent user behavior as evidence of a stable preference. The problem is that the assistant can also shape that behavior. A user may accept recommendations because they match an existing preference, because the model has earned trust, because checking is costly, or because earlier

suggestions have gradually shifted the user’s judgment. The same observed behavior can therefore reflect different underlying causes and should not necessarily produce the same personalization.

Human–AI research shows that expectations about AI reliability can change team behavior and that machine behavior can alter later human strategy (Bansal et al., 2019; Nikolaidis et al., 2017). Persistent personalization makes this identification problem consequential because clarification, correction, and acceptance can be stored as user-specific state and influence future assistance (Liang et al., 2026; Mehri et al., 2026). If a system treats behavior that it helped produce as evidence of an independent preference, its user model can become self-reinforcing.

A central research question is therefore how to distinguish relatively stable user aims from adaptation induced by prior assistance. A system may need evidence collected under different assistance conditions, such as judgments made before recommendations, occasional preference checks, or controlled variation in what information the AI presents. The goal is to estimate whether the same preference would persist under a different history of assistance, so that personalization reflects the user’s aims rather than behavior created by the system itself. A concrete longitudinal test would elicit a user’s judgment before exposing the recommendation, then compare later unassisted judgments across randomized assistance histories. Persistent records should distinguish explicitly stated preferences from behavior inferred under a particular interface or default. Changes in approval and interruption patterns such as those reported by McCain et al. (2026) are useful observations, but cannot alone identify whether preferences, competence, trust, or the burden of checking have changed.

5.6 How Can We Tell Whether a Human–AI System Has Actually Improved Over Time?

Better performance on later tasks does not by itself show that a Human–AI system has developed greater capability. The system may simply use more inference compute, shift more work to the human, encounter easier cases, or benefit from an updated model. Longitudinal evaluation must therefore isolate improvement caused by the history of interaction from improvement caused by additional resources or changes in the task environment.

Existing benchmarks provide useful starting points. PATH-Bench and AgentCL construct controlled histories to measure transfer, retention, interference, forgetting, and plasticity (Yang et al., 2026; Shu et al., 2026), while StreamBench and LifelongAgentBench evaluate continuous and lifelong improvement (Wu et al., 2024; Zheng et al., 2025). Evaluating joint cognitive development requires an additional intervention: experiments must vary how earlier work was divided between the human and the AI and what feedback each side received.

Recent evaluations sharpen what those controls must exclude. ContinualSkillBench compares sequential skill maintenance with independent execution and a context-only baseline; its finding that context-only adaptation is comparable on average to explicit skill maintenance cautions against attributing every sequential gain to transferable skill abstraction (Guan et al., 2026). A two-phase Terminal-Bench study finds that optimizers with strong initial gains can differ in subsequent transfer and continued improvement, and evaluates regression checks as part of the update process (Wang et al., 2026). The latter is a developer-authored technical report under repeatable, verifiable tasks, so its comparative ranking should not be generalized to open-ended deployment. Both studies strengthen the case for testing what an update preserves, where it transfers, and whether another round of experience still helps.

A useful design is to keep later probe tasks and resource budgets fixed while varying the preceding interaction history. Earlier tasks can differ in who performed them, who reviewed them, and what feedback was available. Human or machine learning can be disabled in matched conditions to identify which participant changed. Comparing the same later probes across these histories can reveal whether earlier work organization actually changed later capability.

The machine-side comparison should include fresh-state execution, raw-history reuse, a frozen skill library,

and adaptive artifacts, with identical base-model versions and matched budgets. Probe transfer after removing the original trajectory from active context to distinguish persistent reuse from immediate contextual support. Charge extraction, curation, failed updates, regression tests, and human review to the learning budget; include descendant-agent computation in execution cost. A procedure amortizes earlier work only if its later benefit exceeds these accumulated costs over the evaluated horizon.

On the human side, use delayed unaided probes and standardized takeover tasks, in addition to joint performance. Model upgrades and harness revisions should be logged as interventions, since either can change performance without learning from the team’s own history. For open-ended tasks, keep independent outcome checks separate from the evaluator used to select updates. This design makes negative transfer, supervision dependence, and changes in human readiness visible even when aggregate throughput rises.

The evaluation must also look beyond aggregate accuracy. Two Human–AI systems may achieve the same performance while differing substantially in machine transfer, human readiness to intervene, calibration, or the independence of their errors. Evidence of joint cognitive development therefore requires a causal result: changing the history of how work was organized should produce a measurably different Human–AI system on the same later tasks under comparable resources.

6 Conclusion

Human–AI collaboration should be studied as a process of joint cognitive development. How cognitive work is organized on one task determines the experience available to both the human and the AI. That experience can change human skill and judgment, machine memory and policy, and the coordination that develops between them. The Human–AI system that encounters the next task is therefore partly a product of how earlier work was organized.

Several established literatures become more consequential when viewed through this longer timescale. Human cognition shows how limited cognitive resources are selectively spent and how experience can restructure later cognition. Human–Automation research shows that dividing work between people and machines changes human practice, readiness, and skill. Mixed-initiative and Human–AI research make human involvement itself a dynamic decision. Modern agents add a machine participant that can perform cognitive operations, decide which operations to invoke, and use accumulated experience to change those decisions on later tasks. Together, these literatures describe a system in which the organization of current work helps produce the capabilities available for future work.

Current performance and future capability need not favor the same organization of work. Automating a familiar case may improve efficiency now while reducing the human practice needed to maintain skill. Asking for human review may cost more on the current task while producing supervision that improves later machine behavior. Routing, verification, and personalization can similarly change the experience from which later behavior is learned. Evaluation must therefore ask not only whether performance improves, but how that improvement was produced and what happened to human, machine, and joint capability along the way. Designing Human–AI systems requires reasoning about current performance and future development together.

The resulting research problem is larger than deciding how to use the cognition available at the present moment. It is also a problem of shaping what cognition, skill, judgment, and coordination will be available in the future. How humans and AI work together today changes what each can do tomorrow.

Conflicts of Interest

The authors declare no conflicts of interest.

Use of AI-Assisted Technologies

The authors used OpenAI’s ChatGPT during manuscript preparation to assist with literature search, organization, and language refinement. All content was reviewed and verified by the authors.

References

- Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.
- Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991.
- Falk Lieder and Thomas L. Griffiths. Strategy selection as rational metareasoning. *Psychological Review*, 124(6):762–794, 2017.
- Amitai Shenhav, Matthew M. Botvinick, and Jonathan D. Cohen. The expected value of control: An integrative theory of anterior cingulate cortex function. *Neuron*, 79(2):217–240, 2013.
- Thomas O. Nelson and Louis Narens. Metamemory: A theoretical framework and new findings. In Gordon H. Bower, editor, *The Psychology of Learning and Motivation*, volume 26, pages 125–173. Academic Press, 1990. doi:10.1016/S0079-7421(08)60053-5.
- Lisa K. Son and Janet Metcalfe. Metacognitive and control strategies in study-time allocation. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 26(1):204–221, 2000.
- Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.
- Mica R. Endsley and Esin O. Kiris. The out-of-the-loop performance problem and level of control in automation. *Human Factors*, 37(2):381–394, 1995.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. arXiv:2408.03314, 2024.
- Yan Zhou, Yue Ouyang, Kaiyang Zheng, and Suncheng Xiang. CoBa: Cost-effective test-time scaling via compute-balanced routing. arXiv:2608.07424, 2026.
- Paul M. Fitts, editor. *Human Engineering for an Effective Air-Navigation and Traffic-Control System*. National Research Council, Washington, DC, 1951.
- Thomas B. Sheridan and William L. Verplank. *Human and Computer Control of Undersea Teleoperators*. MIT Man-Machine Systems Laboratory, 1978.
- Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A*, 30(3):286–297, 2000.
- Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 159–166, 1999. doi:10.1145/302979.303030.
- David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems*, volume 31, pages 6147–6157, 2018.

- William G. Chase and Herbert A. Simon. Perception in chess. *Cognitive Psychology*, 4(1):55–81, 1973.
- John R. Anderson. Acquisition of cognitive skill. *Psychological Review*, 89(4):369–406, 1982.
- Gordon D. Logan. Toward an instance theory of automatization. *Psychological Review*, 95(4):492–527, 1988.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911, 2025. <https://proceedings.mlr.press/v267/wang25bx.html>.
- Agamdeep Singh, Srishti Gautam, Priyanshu Gupta, Nikita Mehrotra, Tanmay Bakshi, and Sumit Gulwani. Reason Wide, Not Deep: Amortizing the reasoning premium into distilled skills. arXiv:2608.07885, 2026.
- Yibo Li, Zijie Lin, Ailin Deng, Xuan Zhang, Yufei He, Shuo Ji, Tri Cao, and Bryan Hooi. Just-In-Time Reinforcement Learning: Continual learning in LLM agents without gradient updates. arXiv:2601.18510, 2026.
- Guibin Zhang, Haiyang Yu, Kaiming Yang, Bingli Wu, Fei Huang, Yongbin Li, and Shuicheng Yan. EvoRoute: Experience-driven self-routing LLM agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 38213–38225, 2026. doi:10.18653/v1/2026.acl-long.1771. Also available as arXiv:2601.02695.
- Yaoqi Guo, Ying Xiao, Jie M. Zhang, Mark Harman, Yiling Lou, Yang Liu, and Zhenpeng Chen. EET: Experience-driven early termination for cost-efficient software engineering agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 33008–33024, 2026. doi:10.18653/v1/2026.findings-acl.1652.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, number 17, pages 19632–19642, 2024. doi:10.1609/aaai.v38i17.29936.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023.
- Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41622–41652, 2026. doi:10.18653/v1/2026.findings-acl.2069.
- Sirui Liang, Pengfei Cao, Jian Zhao, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. Learning how to remember: A meta-cognitive management method for structured and transferable agent memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 30733–30753, 2026. doi:10.18653/v1/2026.findings-acl.1535. Also available as arXiv:2601.07470.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents. arXiv:2605.06614, 2026.

- Shicheng Ye and Chao Yu. Joint learning of experiential rules and policies for large language model agents. arXiv:2606.27136, 2026.
- Xidong Yang, Xingyi Zhang, Wenhao Li, Wenyan Liu, Junjie Sheng, Yun Hua, Wei Yin, Tao Fang, Chuyun Shen, and Xiangfeng Wang. PATH-Bench: Path-dependent evaluation of lifelong agents. arXiv:2608.01149, 2026.
- Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. AgentCL: Toward rigorous evaluation of continual learning in language agents. arXiv:2606.02461, 2026.
- Ananth Jonnavittula, Shaunak A. Mehta, and Dylan P. Losey. SARI: Shared autonomy across repeated interaction. *ACM Transactions on Human-Robot Interaction*, 13(2):1–36, 2024. doi:10.1145/3651994.
- Daniel Kahneman and Gary Klein. Conditions for intuitive expertise: A failure to disagree. *American Psychologist*, 64(6):515–526, 2009.
- Gagan Bansal, Besmira Nushi, Ece Kamar, Daniel S. Weld, Walter S. Lasecki, and Eric Horvitz. Updates in Human-AI teams: Understanding and addressing the performance/compatibility tradeoff. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, number 1, pages 2429–2437, 2019. doi:10.1609/aaai.v33i01.33012429.
- Stefanos Nikolaidis, Yu Xiang Zhu, David Hsu, and Siddhartha Srinivasa. Human–robot mutual adaptation in shared autonomy. In *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction*, pages 294–302, 2017. doi:10.1145/2909824.3020252.
- Kaiqu Liang, Julia Kruk, Shengyi Qian, Xianjun Yang, Shengjie Bi, Yuanshun Yao, Shaoliang Nie, Mingyang Zhang, Lijuan Liu, Jaime Fernández Fisac, Shuyan Zhou, and Saghar Hosseini. Learning personalized agents from human feedback. arXiv:2602.16173, 2026.
- Wei Yang, Defu Cao, Jiacheng Pang, Muyan Weng, and Yan Liu. Adaptive collaboration with humans: Metacognitive policy optimization for multi-agent LLMs with continual learning. arXiv:2603.07972, 2026.
- Aditya Aggarwal and Nahid Farhady Ghalaty. Self-improving AI coding agents through accumulated behavioral rules: A closed-loop framework. arXiv:2607.13091, 2026.
- Thomas L. Griffiths, Falk Lieder, and Noah D. Goodman. Rational use of cognitive resources: Levels of analysis between the computational and the algorithmic. *Topics in Cognitive Science*, 7(2):217–229, 2015. doi:10.1111/tops.12142.
- Falk Lieder and Thomas L. Griffiths. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and Brain Sciences*, 43:e1, 2020. doi:10.1017/S0140525X1900061X.
- Frederick Callaway, Shakir Mohamed Gul, Paul M. Krueger, Thomas L. Griffiths, and Falk Lieder. Learning to select computations. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, pages 776–785, 2018. arXiv:1711.06892.
- Mark K. Ho, Zachary Abel, David Correa, Michael L. Littman, Jonathan D. Cohen, and Thomas L. Griffiths. People construct simplified mental representations to plan. *Nature*, 606(7912):129–136, 2022. doi:10.1038/s41586-022-04743-9.
- Asher Koriati. Monitoring one’s own knowledge during study: A cue-utilization approach to judgments of learning. *Journal of Experimental Psychology: General*, 126(4):349–370, 1997. doi:10.1037/0096-3445.126.4.349.

- Janet Metcalfe and Nate Kornell. A region of proximal learning model of study time allocation. *Journal of Memory and Language*, 52:463–477, 2005. doi:10.1016/j.jml.2004.12.001.
- Amitai Shenhav, Sebastian Musslick, Falk Lieder, Wouter Kool, Thomas L. Griffiths, Jonathan D. Cohen, and Matthew M. Botvinick. Toward a rational and mechanistic account of mental effort. *Annual Review of Neuroscience*, 40:99–124, 2017. doi:10.1146/annurev-neuro-072116-031526.
- Julius Froemer, H. C. Lin, C. K. Wolf, Michael Inzlicht, and Amitai Shenhav. Expectations of reward and efficacy guide cognitive control allocation. *Nature Communications*, 12, 2021. doi:10.1038/s41467-021-21315-z.
- David Masis Obando, Sebastian Musslick, and Jonathan D. Cohen. Learning expectations shape cognitive control allocation. *Proceedings of the National Academy of Sciences*, 122(44):e2416720122, 2025. doi:10.1073/pnas.2416720122.
- Walter Schneider and Richard M. Shiffrin. Controlled and automatic human information processing: I. Detection, search, and attention. *Psychological Review*, 84(1):1–66, 1977.
- Richard M. Shiffrin and Walter Schneider. Controlled and automatic human information processing: II. Perceptual learning, automatic attending, and a general theory. *Psychological Review*, 84(2):127–190, 1977.
- Fernand Gobet and Herbert A. Simon. Templates in chess memory: A mechanism for recalling several boards. *Cognitive Psychology*, 31:1–40, 1996.
- Marko Bilalić, Peter McLeod, and Fernand Gobet. Why good thoughts block better ones: The mechanism of the pernicious Einstellung effect in chess. *Cognition*, 108(3):652–661, 2008.
- Erik Hollnagel and Anne Bye. Principles for modelling function allocation. *International Journal of Human-Computer Studies*, 52(2):253–265, 2000. doi:10.1006/ijhc.1999.0288.
- Erik Hollnagel and David D. Woods. *Joint Cognitive Systems: Foundations of Cognitive Systems Engineering*. CRC Press, 2005.
- David B. Kaber and Mica R. Endsley. The effects of level of automation and adaptive automation on human performance, situation awareness and workload in a dynamic control task. *Theoretical Issues in Ergonomics Science*, 5(2):113–153, 2004. doi:10.1080/1463922021000054335.
- Raja Parasuraman and Victor Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997.
- John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004. doi:10.1518/hfes.46.1.50_30392.
- J. C. R. Licklider. Man-computer symbiosis. *IRE Transactions on Human Factors in Electronics*, HFE-1:4–11, 1960.
- Edwin Hutchins. *Cognition in the Wild*. MIT Press, 1995.
- Paul Scerri, David V. Pynadath, and Milind Tambe. Towards adjustable autonomy for the real world. *Journal of Artificial Intelligence Research*, 17:171–228, 2002. doi:10.1613/jair.1037.

- Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7076–7087, 2020.
- Bryan Wilder, Eric Horvitz, and Ece Kamar. Learning to complement humans. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pages 1526–1533, 2020. doi:10.24963/ijcai.2020/212.
- Ruijiang Gao, Maytal Saar-Tsechansky, Maria De-Arteaga, Ligong Han, Wei Sun, Min Kyung Lee, and Matthew Lease. Learning complementary policies for human-AI teams. arXiv:2302.02944, 2023.
- Hussein Mozannar, Arvind Satyanarayan, and David Sontag. Teaching humans when to defer to a classifier via exemplars. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(5):5323–5331, 2022.
- Thodoris Lykouris and Wentao Weng. Learning to Defer in Congested Systems: The AI–Human Interplay. arXiv:2402.12237, 2024. Version 4, updated August 12, 2025. <https://arxiv.org/abs/2402.12237>.
- Shervin Javdani, Mark S. T. Admoni, Andrea Pellegrinelli, Siddhartha S. Srinivasa, and J. Andrew Bagnell. Shared autonomy via hindsight optimization for teleoperation and collaboration. *The International Journal of Robotics Research*, 37(7):717–742, 2018. doi:10.1177/0278364918776060.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi:10.1145/3586183.3606763.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. arXiv:2602.08234, 2026.
- Zhiyuan Fan, Wenwei Jin, Feng Zhang, Bin Li, Yihong Dong, Yao Hu, and Jiawei Li. Evolving-RL: End-to-end optimization of experience-driven self-evolving capability within agents. arXiv:2605.10663, 2026.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. EvolveR: Self-evolving LLM agents through an experience-driven lifecycle. arXiv:2510.16079, 2025.
- Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, Tianyu Huai, Qin Chen, Xin Li, Kai Chen, Bo Zhang, Xipeng Qiu, and Liang He. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. arXiv:2508.19005, 2025.
- Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. StreamBench: Towards benchmarking continuous improvement of language agents. In *Advances in Neural Information Processing Systems*, 37:107039–107063, 2024. doi:10.52202/079017-3398.
- Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM agents as lifelong learners. arXiv:2505.11942, 2025.
- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI agents with reinforcement learning. arXiv:2508.03680, 2025.

- Zhiming Wang, Jinwei He, and Feng Lu. Requesting expert reasoning: Augmenting LLM agents with learned collaborative intervention. arXiv:2602.22546, 2026.
- Shuhaib Mehri, Priyanka Kargupta, Tal August, and Dilek Hakkani-T"ur. Learning user preferences through interaction for long-term collaboration. arXiv:2601.02702, 2026.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. arXiv:2603.25158, 2026.
- Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 16803–16822, 2026. doi:10.18653/v1/2026.findings-acl.829. Also available as arXiv:2512.10696.
- Klein, G. A., Calderwood, R., and Clinton-Cirocco, A. Rapid Decision Making on the Fire Ground: The Original Study Plus a Postscript. *Journal of Cognitive Engineering and Decision Making*, 4(3):186–209, 2010. doi:10.1518/155534310X12844000801203.
- David Silver and Richard S. Sutton. Welcome to the era of experience. Preprint of a chapter to appear in George Konidaris, editor, *Designing an Intelligence*. MIT Press, 2025.
- OpenAI. Research acceleration: The view inside OpenAI. OpenAI, September 6, 2026. Research publication. <https://openai.com/index/research-acceleration-view-inside-openai/>. Accessed September 7, 2026.
- Anthropic. Effective context engineering for AI agents. Anthropic Engineering, September 29, 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Accessed September 7, 2026.
- Anthropic. Equipping agents for the real world with Agent Skills. Anthropic Engineering, October 16, 2025. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>. Accessed September 7, 2026.
- Anthropic. Effective harnesses for long-running agents. Anthropic Engineering, November 26, 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>. Accessed September 7, 2026.
- Anthropic. Scaling Managed Agents: Decoupling the brain from the hands. Anthropic Engineering, April 8, 2026. <https://www.anthropic.com/engineering/managed-agents>. Accessed September 7, 2026.
- Alexandre Moufarek and Adrian Bolton. From Atari to EVE Online: Building on 15 Years of AI Research in Games. Google DeepMind, August 21, 2026. Research agenda. <https://deepmind.google/blog/from-atari-to-eve-online-building-on-15-years-of-ai-research-in-games/>. Accessed September 7, 2026.
- Co-Scientist Team. Co-Scientist: A multi-agent AI partner to accelerate research. Google DeepMind, May 19, 2026. <https://deepmind.google/blog/co-scientist-a-multi-agent-ai-partner-to-accelerate-research/>. Accessed September 7, 2026.
- Ryan Lopopolo. Harness engineering: leveraging Codex in an agent-first world. OpenAI Engineering, February 11, 2026. <https://openai.com/index/harness-engineering/>. Accessed September 7, 2026.

- Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta Context Engineering via Agentic Skill Evolution. arXiv:2601.21557, 2026. <https://arxiv.org/abs/2601.21557>. Accessed September 7, 2026.
- Seth Karten, Alex L. Zhang, Kevin Thomas, Sebastian Müller, Elie Bakouch, Daniel Auras, Mika Senghaas, Fares Obeid, Konstantin Dunas, Johannes Hagemann, and Sami Jaghouar. Prime Agent: A Self-Improving RLM Harness. arXiv:2608.23552, 2026. Technical report. <https://arxiv.org/abs/2608.23552>. Accessed September 7, 2026.
- Prithvi Rajasekaran. Harness design for long-running application development. Anthropic Engineering, March 24, 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>. Accessed September 7, 2026.
- Judy Hanwen Shen and Alex Tamkin. How AI Impacts Skill Formation. arXiv:2601.20245, 2026. <https://arxiv.org/abs/2601.20245>. Accessed September 7, 2026.
- Miles McCain, Thomas Millar, Saffron Huang, Jake Eaton, Kunal Handa, Michael Stern, Alex Tamkin, Matt Kearney, Esin Durmus, Judy Shen, Jerry Hong, Brian Calvert, Jun Shern Chan, Francesco Mosconi, David Saunders, Tyler Neylon, Gabriel Nicholas, Sarah Pollack, Jack Clark, and Deep Ganguli. Measuring AI agent autonomy in practice. Anthropic Research, February 18, 2026. Observational study. <https://www.anthropic.com/research/measuring-agent-autonomy>. Accessed September 7, 2026.
- Don Wallace, Conor Griffin, Sean O’Neill, Thang Luong, and Owen Larter. Conjecture Machines: AI agents and the new validation bottleneck in science. Google DeepMind, July 2026. Policy essay. <https://deepmind.google/public-policy/conjecture-machines-ai-agents-and-the-new-validation-bottleneck-in-science/>. Accessed September 7, 2026.
- Tianyi Guan, Yiding Wang, Haotong Yang, Siyuan Cao, Shirui Liu, Yi Hu, Jiaqi Li, and Muhan Zhang. ContinualSkillBench: Can LLM Agents Truly Evolve Their Capabilities? arXiv:2608.03874, 2026. <https://arxiv.org/abs/2608.03874>. Accessed September 7, 2026.
- Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. Do Agent Optimizers Compound? A Continual-Learning Evaluation on Terminal-Bench 2.0. arXiv:2607.14004, 2026. RELAI technical report. <https://arxiv.org/abs/2607.14004>. Accessed September 7, 2026.